

WEEKLY PROGRESS REPORT -04

CLIENT & FREELANCER DASHBOARDS, JOB POSTING, ONBOARDING & AUTH FLOWS (NO DATABASE CONNECTED)

Project: MuktoKormo (Freelance Marketplace)	Scope: Dashboard & Workflow Frontend	Sprint: Week 4 (Multi-Role UX)
Frontend: Next.js 16, React 19, Tailwind v4	Database: Not Connected (Mock State)	Next Phase: Database & Backend Wiring
Team Members	Abdul Ahad Akid (2207018)	Ashraful Hosen (2207042)

1. Project Overview & Current Architectural Scope

Executive Context: MuktoKormo is an Upwork/Fiverr-grade freelance marketplace engineered for the Bangladeshi digital ecosystem, connecting local employers with verified remote talent through a milestone-based escrow workflow.

Sprint Objectives: This sprint converted the platform from a single public landing surface into a genuine multi-role product. Two dedicated dashboard workspaces, a guided job-posting pipeline, a fully reimagined onboarding system, and a role-aware authentication entry point were designed and implemented from scratch on top of the existing frontend foundation.

Current Architectural Status (Frontend-Only): Every surface described in this report operates purely on the client side using React 19 state hooks and structured mock datasets. No backend database is connected. Checklist progression, wizard step validation, fee and budget logic, OTP cooldowns, and role transitions all simulate real platform behaviour, establishing a battle-tested UI/UX foundation that can be wired to persistence without redesign.

Sprint Scope: The sprint covered the two role workspaces, the job-posting pipeline, the onboarding system, the authentication entry point, and the shared component and data layers underpinning them. Explicitly out of scope were server infrastructure, third-party payment certification, real messaging, and notification delivery, all of which remain tracked in the integration plan.

2. Work Completion — Dual-Role Dashboard Workspaces

A. Client Workspace (/dashboard/client): A persistent onboarding checklist spanning profile completion, identity verification, payment method setup, and first job posting, backed by four KPI statistic cards, an upcoming-consultation call panel, and a saved-draft tracker. B. Client Contracts & Payments: An active contract table exposing milestone status, escrow release state, counterparty details, and a BDT payment history ledger. C. Client Resources Hub: Curated guides, platform policy documents, and announcement cards presented as reusable resource tiles.	D. Freelancer Workspace (/dashboard/freelancer): A wallet and escrow balance summary, a dynamic profile-strength meter with actionable gaps, and a personalised recommended-jobs feed. E. Freelancer Contracts & Proposals: A contract pipeline with proposal status tracking alongside a verification checklist covering NID, phone number, and skill proof. F. Freelancer Resources Hub: Curated learning material, template downloads, and platform announcements in a dedicated resource view.
--	--

3. Work Completion — Five-Step Job Posting Pipeline (/post-job)

- Guided Five-Step Wizard:** A sequential pipeline spanning job title, skill requirements, engagement scope, budget, and description, driven by a persistent progress indicator and strict per-step validation that blocks advancement until each stage is satisfied.
- Skill Selection Stage:** A searchable skill picker with dynamic tagging pills and explicit proficiency tagging, reusing the same skill taxonomy as the onboarding flow to keep the data model consistent.
- Scope & Budget Stage:** An engagement-type selector separating Fixed Price from Hourly work, paired with a BDT budget range input validated against the platform's established tier boundaries.
- Description & Preview Stage:** A rich requirement entry area, an attachment placeholder, and a final consolidated review panel presented to the client before simulated submission.
- State & Validation:** Step index, field values, and inline error messages are managed through React state hooks, keeping the entire flow free of backend calls.
- Deep-Linkable Step State:** The active stage is expressed in the URL, so a client can return to a specific step of an in-progress draft rather than being forced back to the beginning of the wizard.

4. Work Completion — Reimagined Multi-Step Onboarding (/signup)

- **Client Onboarding Flow:** Organization-versus-individual account type selection, company profile capture, and structured team member records collected as a guided sequence.
- **Freelancer Account & Resume Flow:** Identity capture followed by a professional headline, work experience timeline, and education history assembled into a live profile preview.
- **Skills & Profile Stage:** Dynamic skill tagging with an explicit proficiency level plus an hourly rate input denominated in BDT.
- **Role Selection Stage:** Visual determination cards ('I'm a Client' vs 'I'm a Freelancer') that drive the entire downstream step tree rather than acting as a decorative control.
- **Progress & Terms:** A step progress bar, per-step validation, and terms acceptance are handled entirely on the client, with each stage addressable directly for review and correction.

5. Work Completion — Role-Aware Authentication Entry (/login)

- **Dual-Role Login Surface:** A seamless client-side switcher between the Client (Employer) and Freelancer workspaces that reframes the surrounding copy, imagery, and available actions.
- **Bangladeshi SMS OTP Authentication:** Phone sign-in pre-filled with the +880 country code, a six-digit verification stage, and a sixty-second resend cooldown timer.
- **Credential & Social Entry:** An email and password form with a password visibility toggle, complemented by Google and Apple sign-in affordances.
- **Role-Based Routing:** On successful sign-in the session state resolves a role and redirects to the matching workspace, replacing the previous single-destination behaviour.

6. Design Language & Interface Standards

A. Visual Language: An emerald-based palette paired with a neutral slate scale, applied consistently so that every newly built surface reads as part of one product rather than a collection of separate pages.

B. Component Language: A consistent rounded-card system, uniform spacing rhythm, and a shared status vocabulary for badges, pills, and progress indicators.

C. Localisation Conventions: Bangla and English copy coexist across the interface, with all monetary values formatted consistently in BDT and dates rendered through a shared relative-time helper.

D. Responsive Behaviour: Every workspace collapses cleanly to tablet and mobile widths, with secondary navigation re-flowing rather than being simply hidden.

7. Reusable Frontend Architecture & Design System

A. Dashboard Shell: A shared responsive frame providing the sidebar, top bar, and content scaffolding reused by both workspaces so client and freelancer surfaces stay structurally consistent.

B. Job Post Wizard Engine: A self-contained step engine owning the step index, per-step validation, and navigation guards for /post-job.

C. Onboarding Step Modules: Dedicated client organisation and freelancer resume/skills step components, each independently addressable and composable into the /signup step tree.

D. Typed Data Layer: Central TypeScript interfaces plus a mock dataset module and shared formatting helpers (BDT currency, dates, relative time) that isolate presentation from future API payloads.

8. Quality Assurance & Verification

A. Type Safety: The full TypeScript check (tsc --noEmit) passes with no errors across every route and component introduced this sprint.

B. Production Build: A full Next.js production build completes successfully, prerendering every new route without runtime errors.

C. Route Smoke Tests: All eight primary routes were served and verified to return HTTP 200, including deep links into the onboarding step tree.

D. Outstanding Items: Five pre-existing lint errors remain in legacy modal and footer components from earlier sprints; they are unrelated to this surface and are queued for cleanup.

9. Work In Progress (WIP) & Database Integration Plan

- **WIP-1: Database Connection & Migrations:** Provision a PostgreSQL instance and apply the Prisma schema covering User, Profile, Organization, Job, Proposal, Contract, Milestone, Transaction, and Review entities, followed by an initial seed pass mirroring the current mock datasets.
- **WIP-2: Server-Side Authentication Engine:** Replace client-side mock auth with persistent session management (NextAuth.js / JWT), proper password hashing, and an active BD SMS OTP gateway with rate limiting.
- **WIP-3: Workflow Persistence:** Persist onboarding step data, checklist progress, job drafts, and job-posting wizard submissions to backend REST or Server Action endpoints, with optimistic UI updates already anticipated by the current state model.
- **WIP-4: Live MFS Escrow Gateway:** Transition the frontend escrow views into live bKash, Nagad, and Rocket merchant sandbox APIs with IPN callback webhooks and reconciliation of milestone release states.

10. Next Sprint Objectives

- **Wire the existing mock datasets to live Prisma queries**, replacing each static module with server-side data access while preserving the current component contracts so no UI layer needs to be rewritten.
- **Promote authentication to a real session layer**, removing the client-side role resolution in favour of server-verified claims, password hashing, and a live +880 SMS OTP gateway.
- **Ship the end-to-end hiring loop**, connecting job posting, proposal submission, contract award, and milestone escrow into one continuous flow rather than four isolated screens.
- **Activate the live MFS payment gateway**, connecting bKash, Nagad, and Rocket merchant sandbox endpoints with IPN callback reconciliation for escrow release.
- **Harden the release pipeline**, clearing the residual lint backlog in the legacy modal and footer components and adding automated regression coverage for the multi-step flows.
- **Commission real-device usability testing** across the client and freelancer workspaces to validate the density and hierarchy of the dashboard information architecture before database linkage.
- **Introduce a notification and messaging layer** covering proposal updates, contract state changes, and milestone events, which the current static dashboards surface as placeholders only.
- **Formalise the escrow dispute flow** so that a milestone can be contested, reviewed, and resolved rather than being released unilaterally.

11. Conclusion & Handover Notes

Summary of Position: The sprint closed with every planned frontend surface delivered, type-checked, and building cleanly. The presentation layer of MuktoKormo is now complete enough that the remaining risk is concentrated entirely in the data and authentication layers rather than in the interface itself.

Recommendation: Proceeding directly to PostgreSQL and Prisma integration is advised, since the typed data layer introduced this sprint was deliberately shaped around the target entity model. That alignment should allow persistence to be added behind the existing component contracts without visual rework, keeping the current design language intact while the data layer is replaced underneath it.

Risk Note: The only material dependency outside the team's control is certification for the live mobile financial services gateway, which is a lead-time item. It is therefore recommended that the authentication and persistence tracks proceed first, so that escrow integration can begin the moment merchant credentials are issued.